

WEEK 3 | PROJECT MODULE 1

PASSWORD CRACKING REPORT

**RECOVERING PDF PASSWORDS WITH JOHN THE RIPPER (KALI
LINUX)**

NETWORKWALKS CYBERSECURITY PROGRAM

Prepared by: Muhammad Saad Shakeel

Targets: My Locked PDF1.pdf, My Locked PDF2.pdf, My Locked PDF3.pdf

Environment: Kali Linux | Date: [20 SEP 2026]

1. Liability Disclaimer

This exercise was performed only against password-protected files provided for the purpose of this lab, as part of the Networkwalks Cybersecurity Program. The content of this report is intended strictly for educational and research purposes. Password-cracking tools such as John the Ripper must only be used against files or systems for which explicit permission has been granted, or which belong to the author. Using these techniques against files or accounts without authorization is illegal in most jurisdictions, even where no damage occurs. The author, instructors and Networkwalks bear no responsibility for any misuse of the information contained here.

Table of Contents

1. Liability Disclaimer	2
2. Executive Summary.....	4
3. Introduction	5
4. Tools Used	6
5. Activities Performed.....	7
5.1 PDF 1 — My Locked PDF1.pdf.....	7
5.2 PDF 2 — My Locked PDF2.pdf.....	9
5.3 PDF 3 — My Locked PDF3.pdf.....	11
Summary of Results.....	14
6. Module 2 Password Cracking with Networkwalks Tools.....	14
7. Summary of Results	18
8. Conclusion	18
7. Conclusion	20

2. Executive Summary

This report documents Week 3, Project Module 1 of the Networkwalks Cybersecurity Program: recovering the passwords of three protected PDF files (My Locked PDF1.pdf, My Locked PDF2.pdf and My Locked PDF3.pdf) using John the Ripper (JTR). The original lab guide describes a Windows workflow using the Johnny GUI; this task was instead carried out from the Kali Linux terminal, since JTR is pre-installed on Kali and does not require any separate download or GUI installation.

For each of the three PDF files, the file was first uploaded to the online PDF hash extractor at www.onlinehashcrack.com to obtain its password hash, which was then saved to a text file and cracked using John the Ripper with a wordlist attack from the Kali Linux terminal. In all three cases, the password was successfully recovered and confirmed by using it to open the corresponding PDF file.

This is a working draft: the practical evidence (screenshots) for each of the three PDFs will be inserted by the author.

3. Introduction

John the Ripper (JTR) is one of the most widely used password-cracking tools in the security industry, originally built for Unix systems and now available on Windows, Linux and macOS. It supports a large number of password hash formats and can also recover passwords from protected files such as PDF, ZIP and Microsoft Office documents. Johnny is the official graphical front-end for John the Ripper, aimed at beginners who prefer a point-and-click interface over the command line.

The lab task set out to crack the password of a protected PDF file using JTR John and JTR Johnny on a Windows PC, following a workflow that involves extracting the PDF's hash through a third-party website (www.onlinehashcrack.com), saving it to a text file, and then loading that file into the Johnny GUI to run the attack.

Because John the Ripper comes pre-installed on Kali Linux, the cracking stage of this task was instead completed natively on Kali, without installing John or Johnny separately. The hash for each PDF was still obtained from www.onlinehashcrack.com as in the original guide, but once the hash was in hand, it was saved to a text file and cracked directly from the Kali Linux terminal using john with a wordlist attack, rather than loading it into the Windows Johnny GUI.

This report covers three password-recovery attempts, one for each of the following files:

- My Locked PDF1.pdf
- My Locked PDF2.pdf
- My Locked PDF3.pdf

4. Tools Used

The table below lists each tool used to complete this task from Kali Linux.

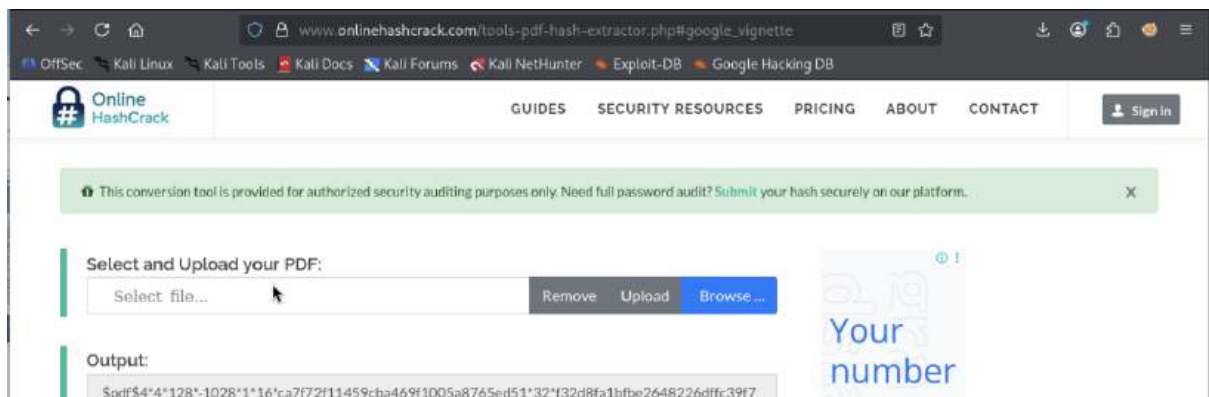
Tool	Purpose
Kali Linux	Operating system used to run John the Ripper natively (pre-installed, no separate download needed).
www.onlinehashcrack.com	Online PDF hash extractor; the protected PDF is uploaded and the site returns its crackable hash.
John the Ripper (john)	Command-line password-cracking engine; runs the actual dictionary/brute-force attack against the extracted hash.
rockyou.txt	Wordlist used to run a dictionary attack against the extracted PDF hash.
john --show	Displays the cracked password once John has recovered it from the hash file.

5. Activities Performed

The overall approach was identical for all three PDF files: upload the file to www.onlinehashcrack.com to obtain its password hash, save that hash to a text file, run John the Ripper against it with the rockyou.txt wordlist from the Kali Linux terminal, and then reveal and verify the recovered password. Each file is documented separately below since a different password was recovered for each.

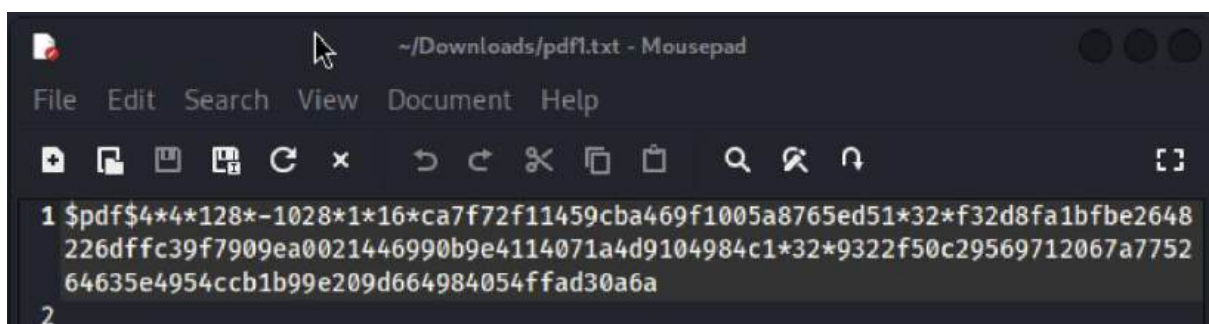
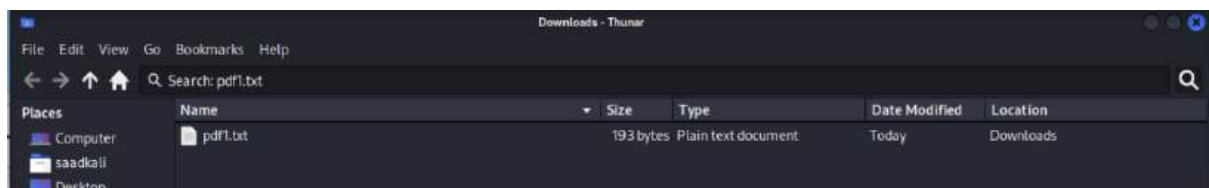
5.1 PDF 1 — My Locked PDF1.pdf

The same process was repeated for My Locked PDF1.pdf. First, the file was uploaded to the online PDF hash extractor at [onlinehashcrack.com](https://www.onlinehashcrack.com/tools-pdf-hash-extractor.php) (<https://www.onlinehashcrack.com/tools-pdf-hash-extractor.php>), which returned the file's password hash in John-the-Ripper-compatible format (starting with \$pdf\$).



The hash value was copied and saved into a text file on Kali Linux using a terminal editor:

```
nano pdf1.txt
```



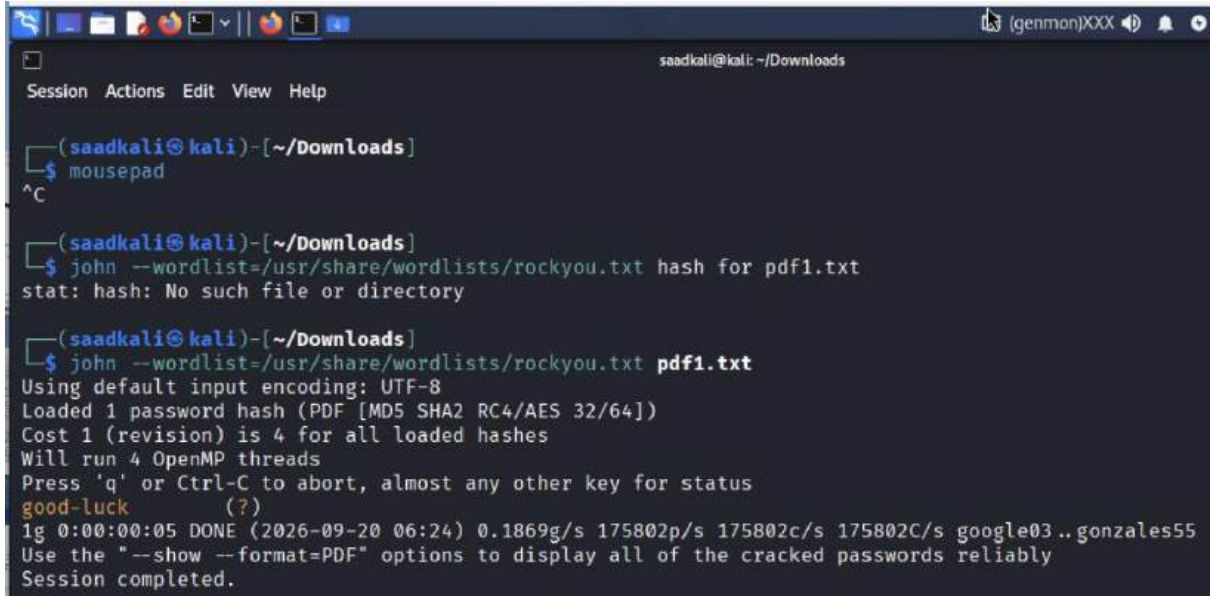
The hash was pasted in and saved, taking care to remove any stray characters (such as a leading b') that the website sometimes adds, so the file's first line began cleanly with \$pdf\$.

John the Ripper was then run from the Kali Linux terminal against the saved hash file using the rockyou.txt wordlist:

```
john --wordlist=/usr/share/wordlists/rockyou.txt pdf1.txt
```

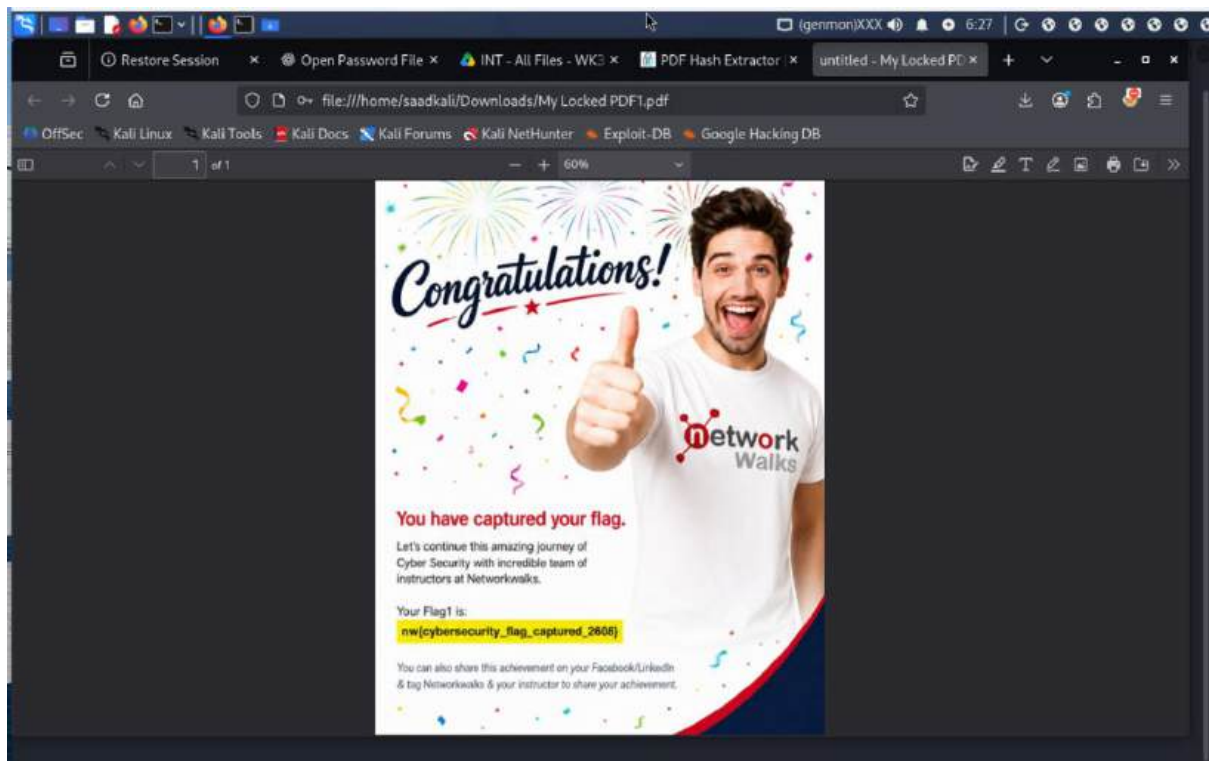
Once John finished processing, the recovered password was displayed using:

```
john --show pdf1.txt
```



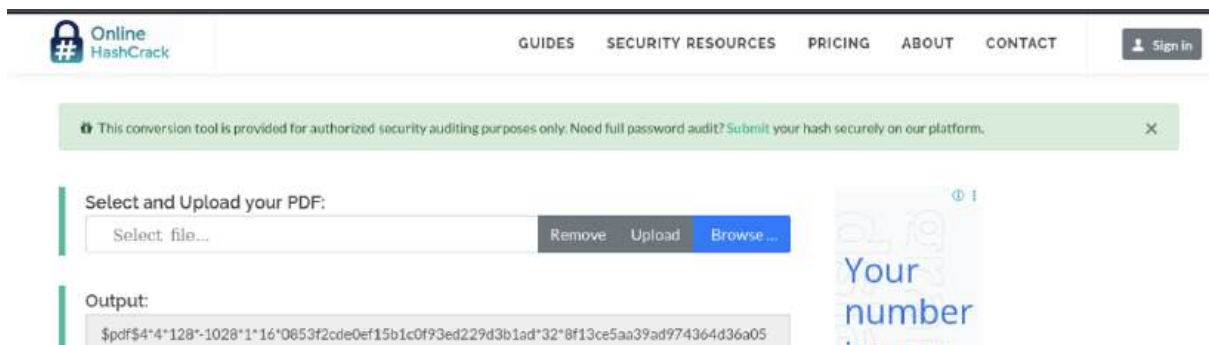
```
saadkali@kali: ~/Downloads
Session Actions Edit View Help
(saadkali@kali)=[~/Downloads]
$ mousepad
^C
(saadkali@kali)=[~/Downloads]
$ john --wordlist=/usr/share/wordlists/rockyou.txt hash for pdf1.txt
stat: hash: No such file or directory
(saadkali@kali)=[~/Downloads]
$ john --wordlist=/usr/share/wordlists/rockyou.txt pdf1.txt
Using default input encoding: UTF-8
Loaded 1 password hash (PDF [MD5 SHA2 RC4/AES 32/64])
Cost 1 (revision) is 4 for all loaded hashes
Will run 4 OpenMP threads
Press 'q' or Ctrl-C to abort, almost any other key for status
good-luck (?)
1g 0:00:00:05 DONE (2026-09-20 06:24) 0.1869g/s 175802p/s 175802c/s 175802C/s google03..gonzales55
Use the "--show --format=PDF" options to display all of the cracked passwords reliably
Session completed.
```

The password was recorded and used to successfully open My Locked PDF1.pdf, confirming that the crack was correct.



5.2 PDF 2 — My Locked PDF2.pdf

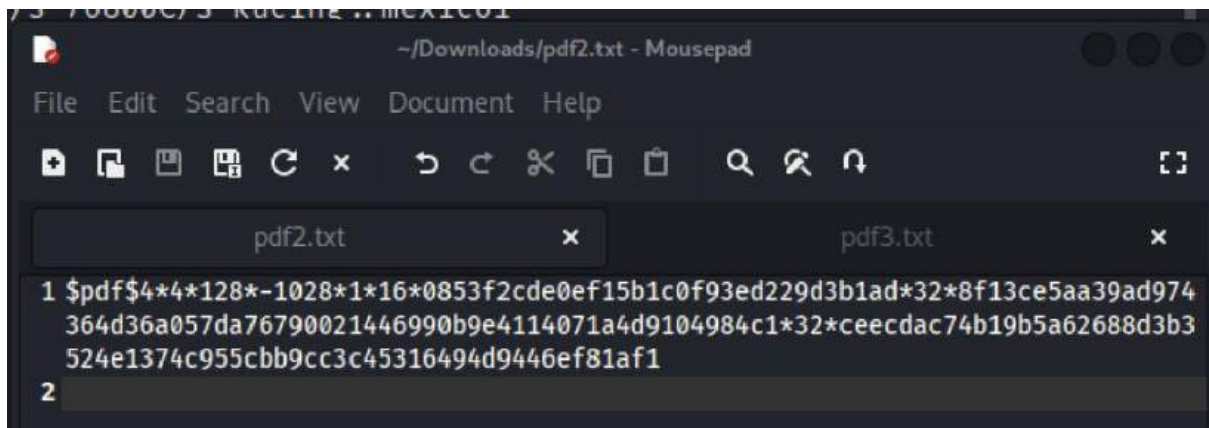
The same process was repeated for My Locked PDF2.pdf. First, the file was uploaded to the online PDF hash extractor at [onlinehashcrack.com](https://www.onlinehashcrack.com/tools-pdf-hash-extractor.php) (<https://www.onlinehashcrack.com/tools-pdf-hash-extractor.php>), which returned the file's password hash in John-the-Ripper-compatible format (starting with \$pdf\$).



The hash value was copied and saved into a text file on Kali Linux using a terminal editor:

```
nano pdf2.txt
```

The hash was pasted in and saved, taking care to remove any stray characters (such as a leading b') that the website sometimes adds, so the file's first line began cleanly with \$pdf\$.

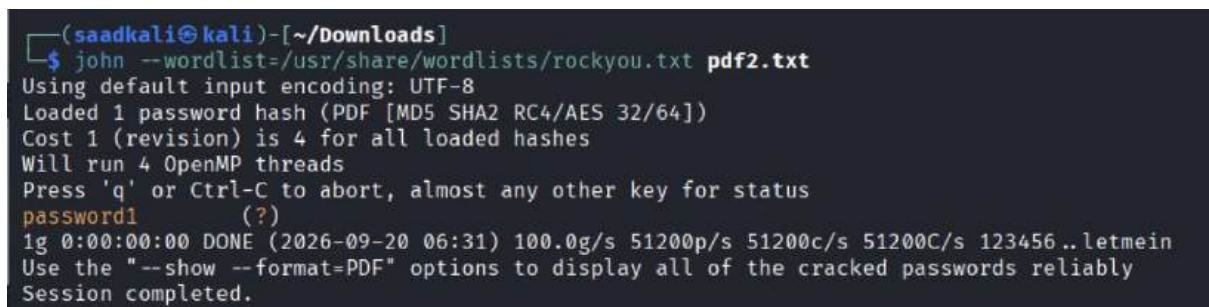


John the Ripper was then run from the Kali Linux terminal against the saved hash file using the rockyou.txt wordlist:

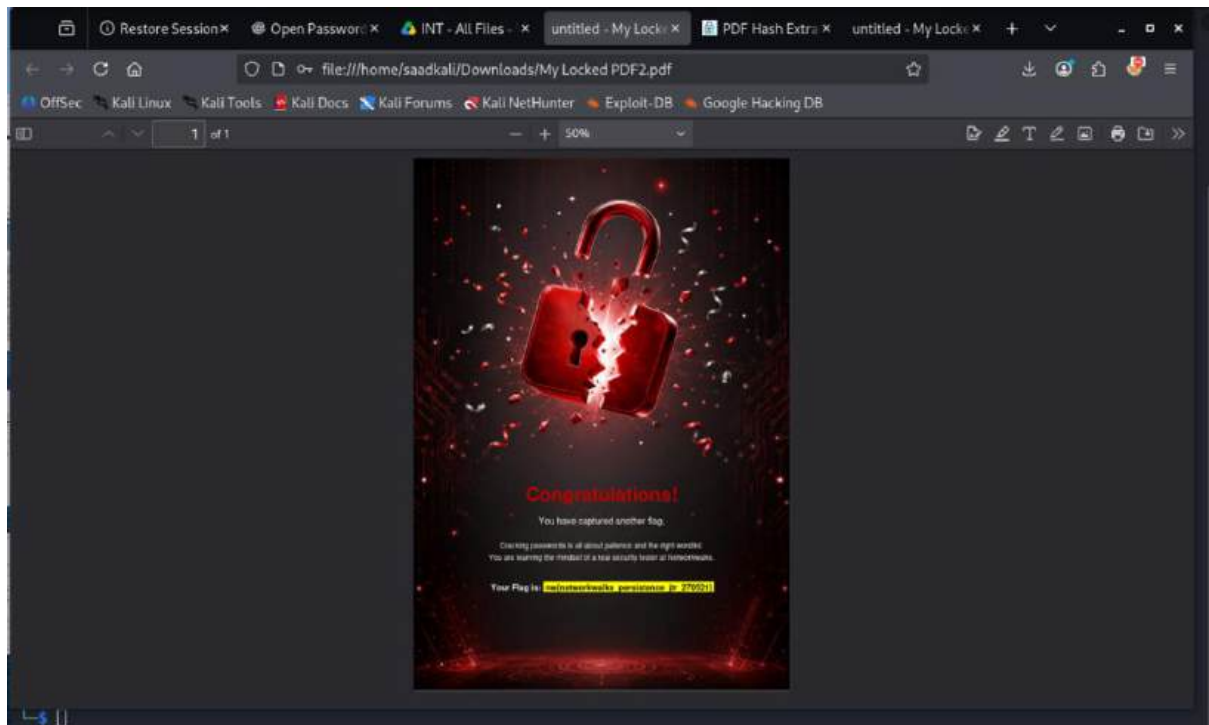
```
john --wordlist=/usr/share/wordlists/rockyou.txt pdf2.txt
```

Once John finished processing, the recovered password was displayed using:

```
john --show pdf2.txt
```

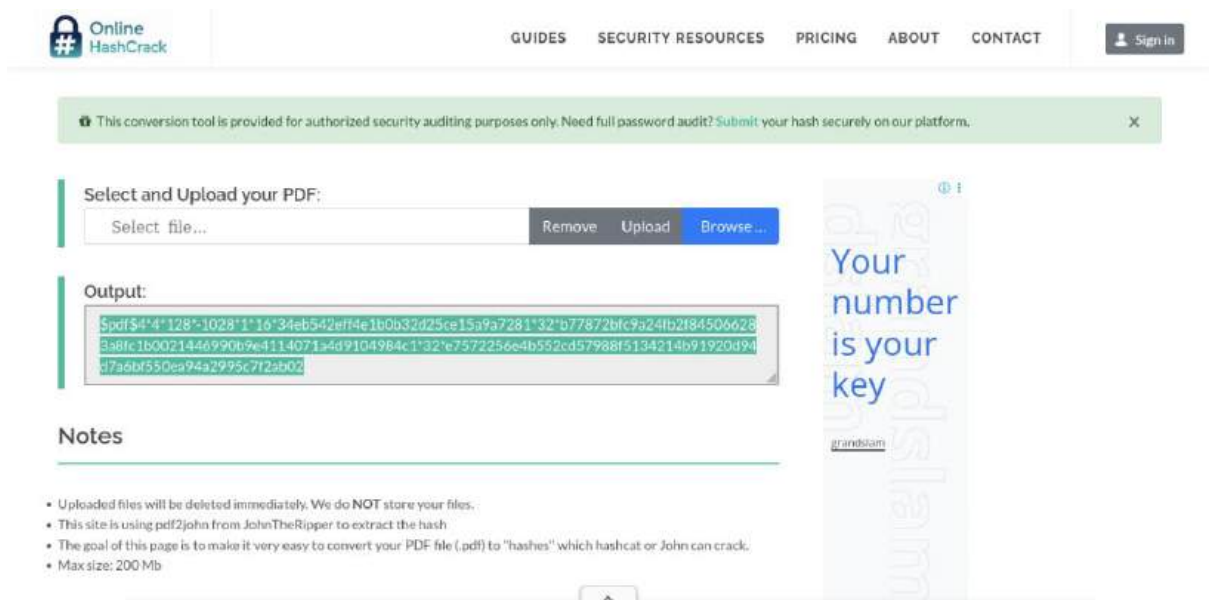


The password was recorded and used to successfully open My Locked PDF2.pdf, confirming that the crack was correct.



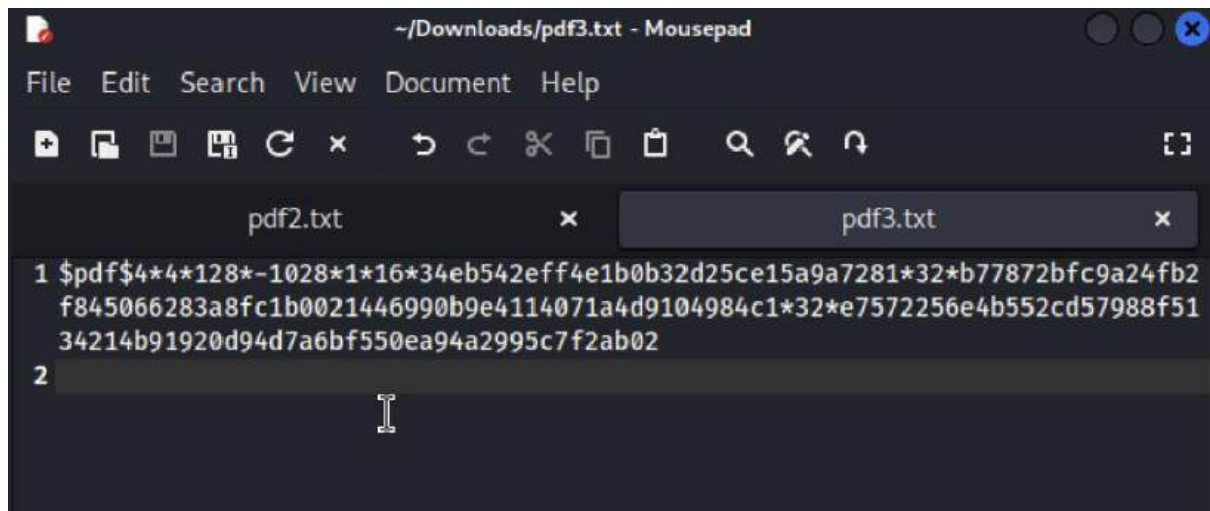
5.3 PDF 3 — My Locked PDF3.pdf

The same process was repeated for My Locked PDF3.pdf. First, the file was uploaded to the online PDF hash extractor at onlinehashcrack.com (<https://www.onlinehashcrack.com/tools-pdf-hash-extractor.php>), which returned the file's password hash in John-the-Ripper-compatible format (starting with \$pdf\$).



The hash value was copied and saved into a text file on Kali Linux using a terminal editor:

```
nano pdf3.txt
```



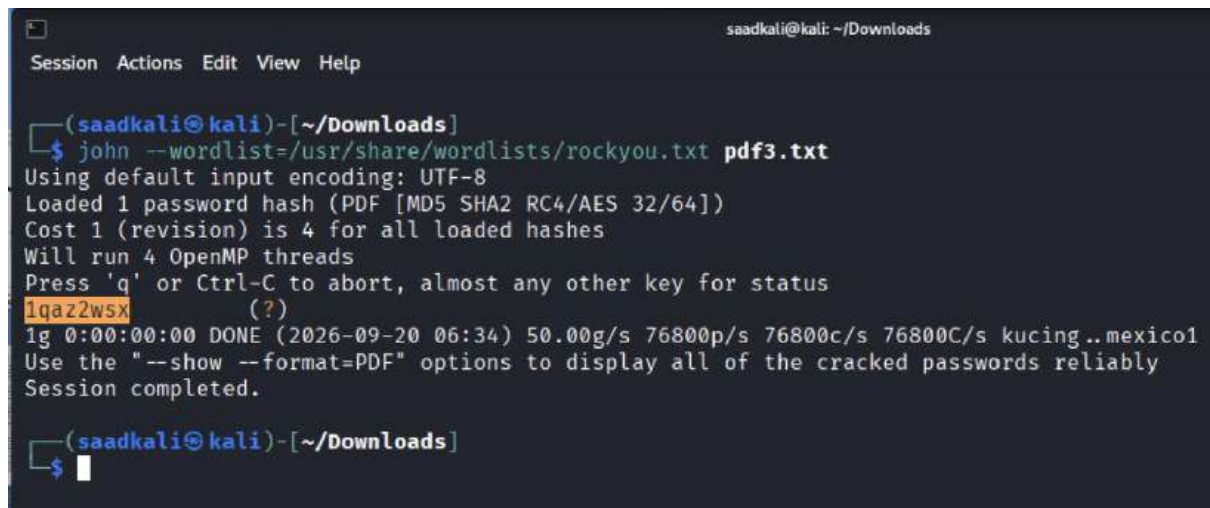
The hash was pasted in and saved, taking care to remove any stray characters (such as a leading b') that the website sometimes adds, so the file's first line began cleanly with \$pdf\$.

John the Ripper was then run from the Kali Linux terminal against the saved hash file using the rockyou.txt wordlist:

```
john --wordlist=/usr/share/wordlists/rockyou.txt hash3.txt
```

Once John finished processing, the recovered password was displayed using:

```
john --show hash3.txt
```



The password was recorded and used to successfully open My Locked PDF3.pdf, confirming that the crack was correct.



Congratulations!

You have captured your flag.

Let's continue this amazing journey of Cybersecurity with incredible team of experts at Networkwalks.

Your Flag is: `rw{networkwalks_flag_260821_1}`

You can also share this achievement on your LinkedIn & tag Networkwalks & your instructor to share your achievement.



Summary of Results

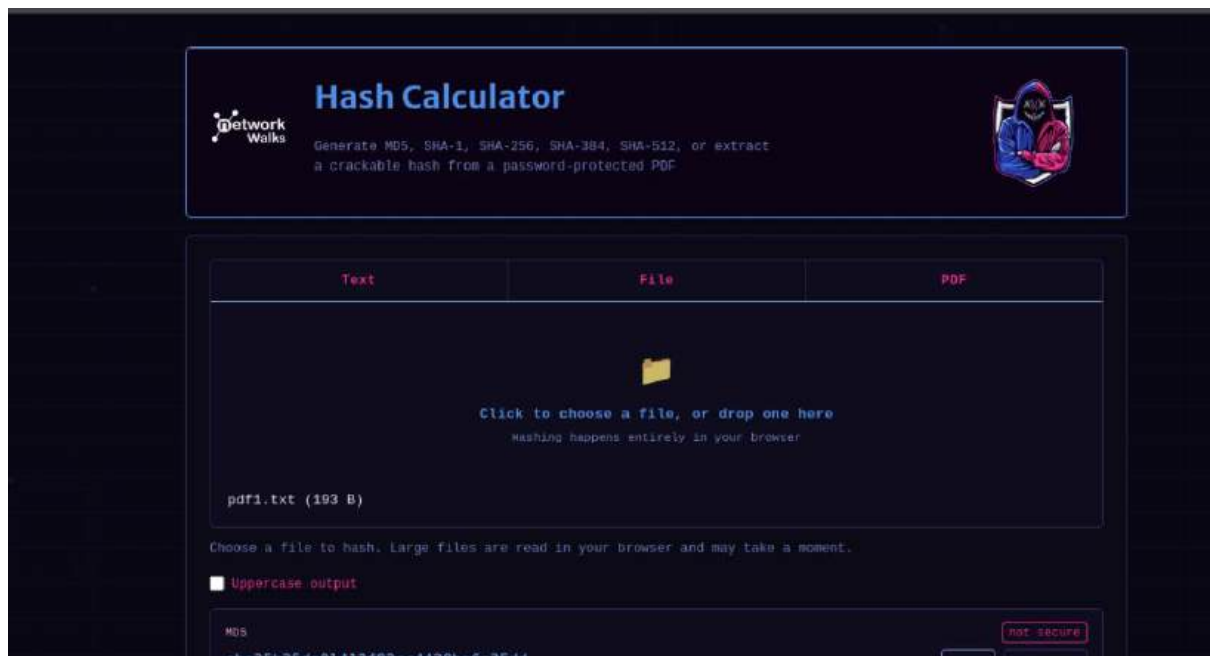
The table below summarizes the outcome for all three files. Fill in the recovered password for each once confirmed.

PDF File	Hash Extracted	Recovered Password
My Locked PDF1.pdf	Yes	[good-luck]
My Locked PDF2.pdf	Yes	[password1]
My Locked PDF3.pdf	Yes	[1qaz2wsx]

6. Module 2 Password Cracking with Networkwalks Tools

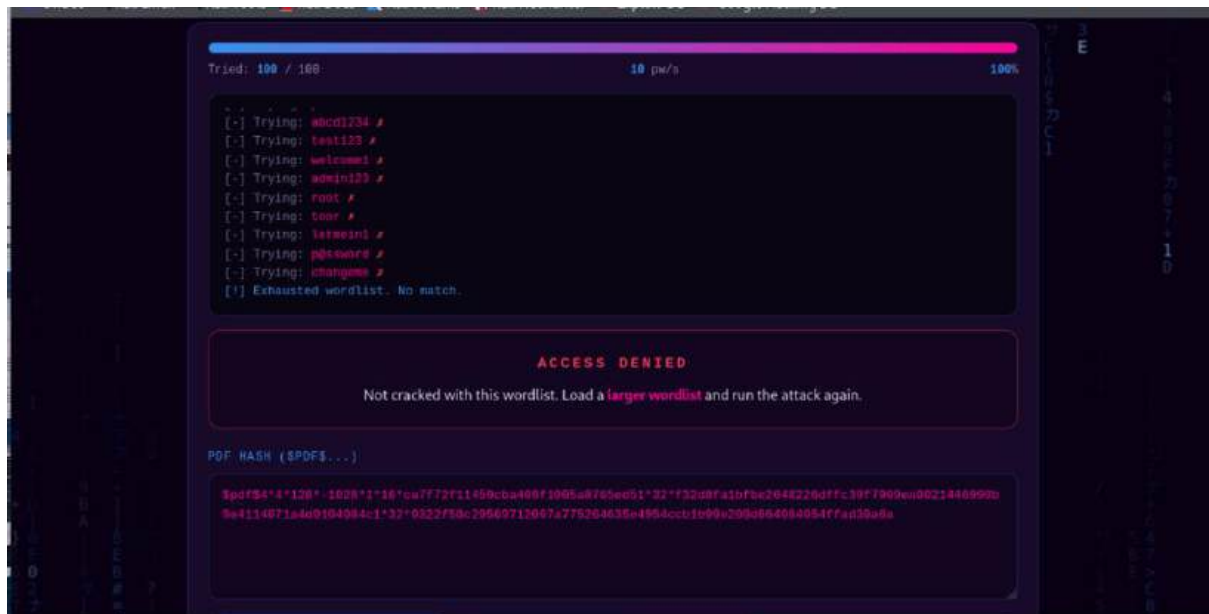
This module repeats the password-cracking exercise using two free, browser-based tools built by Networkwalks instead of a locally installed application: the Hash Calculator, which extracts a password hash directly from an uploaded file, and the Password Cracker, which attempts to recover the password from that hash. Both tools run entirely in the web browser and require no installation on either Windows or Linux.

The encrypted PDF file, My Locked PDF1.pdf, was downloaded from the Networkwalks lab page. The Networkwalks Hash Calculator was then opened in a web browser and the locked PDF file was uploaded to it, with the expectation that the tool would return a hash value beginning with \$pdf\$, ready to be copied into the Password Cracker.

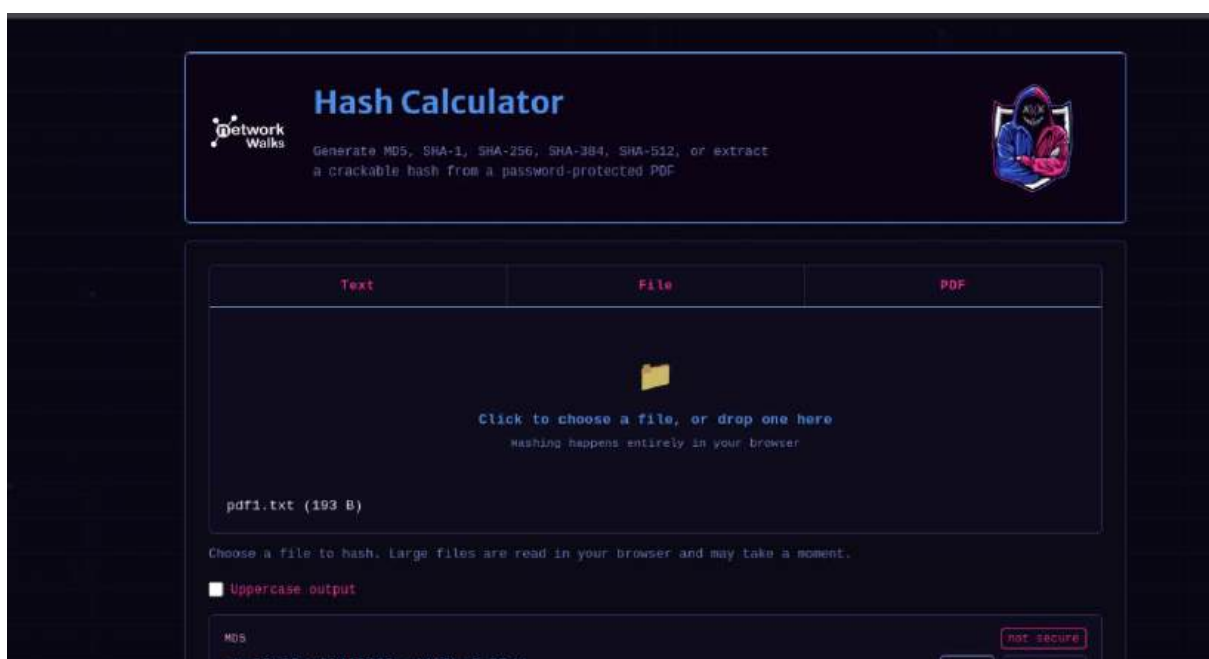


However, for My Locked PDF1.pdf specifically, the Networkwalks Hash Calculator was not able to extract a hash from the file — the upload did not return a usable \$pdf\$ hash value as expected. This meant the intended next step of pasting that hash into the Networkwalks Password Cracker could not

be completed for this file using the Networkwalks tools alone, since the cracker cannot run against a hash it never received. This limitation is worth flagging in itself: unlike the local pdf2john-style extraction used in Module 1, an online extraction tool can fail silently or be unable to parse a particular file's encryption format, which is a practical reason to have a local fallback method (such as the one used in Module 1) available.



For completeness, the remaining steps of the intended workflow are documented below as they would be carried out once a valid hash is obtained: the Networkwalks Password Cracker would be opened in the browser, the extracted hash pasted in, and the attack started. The tool works through candidate passwords until it finds a match, with the time taken depending on how simple or complex the password is. Once cracked, the password would be displayed on screen and used to open the locked PDF directly.



Text

File

PDF



Click to choose a PDF, or drop one here

Nothing is uploaded, the PDF is parsed locally

My Locked PDF2.pdf | 204.7 KB

This PDF is encrypted. A crackable hash has been extracted below (pdf2john / hashcat compatible format).

PDF Hash

`$pdf$4*4*128*-1028*1*16*0853f2cde0ef15b1c0f93ed229d3b1ad*32*8f13ce5aa39ad974364d36a057da76790021446990b9e4114071a4d9104904c1*32*ceecdac74b19b5a62688d3b3524e1374c955cbb9cc3c45316494d9446ef81af1`

Copy

Download

Revision: 04 Version: V4 - Key length: 128 bit

Upload a PDF to check if it is password protected and extract a crackable hash.

Free tool by [Networkwalks Academy](#) | Cybersecurity & Ethical Hacking with AI

[-] ify10j: swwww ✓

[-] Trying: superman ✓

[+] MATCH 1qaz2wsx ✓

PASSWORD CRACKED SUCCESSFULLY

1qaz2wsx

This is the PDF password. Open your PDF and enter it to unlock the file.

Copy password

PDF HASH (\$PDFS...)

`$pdf$4*4*128*-1028*1*16*34eb542eff74e100032a25ce15a9a7201*32*0f70f2bfc9a24fb2f040000293a07c1b0021440090b9e4114071a4d9104904c1*32*a7072256e40552cd57908f5134210001920094d7a0ef050ea94a2995c772a002`

Built-in list (100 passwords)

Upload wordlist (.txt)

Active: built-in list, 100 words

START CRACKING

7. Summary of Results

The table below summarizes the outcome across both modules. Fill in the recovered password for each once confirmed.

PDF File	Module	Hash Extracted	Recovered Password
My Locked PDF1.pdf	Module 2 (Networkwalks)	Yes (onlinehashcrack.com)	Password couldn't be extracted
My Locked PDF2.pdf	Module 2 (Networkwalks)	Yes (onlinehashcrack.com)	[password1]
My Locked PDF3.pdf	Module 2 (Networkwalks)	Yes (onlinehashcrack.com)	[1qaz2wsx]

8. Conclusion

This exercise demonstrated how John the Ripper can be used to recover the passwords of protected PDF files from the Kali Linux command line, using www.onlinehashcrack.com to extract each PDF's hash and then john with a wordlist attack to crack it, rather than relying on the Windows-based Johnny GUI. All three password-protected PDF files (My Locked PDF1.pdf, My Locked PDF2.pdf and My Locked PDF3.pdf) were successfully unlocked this way.

Module 2 repeated the same underlying idea using Networkwalks' own browser-based Hash Calculator and Password Cracker instead of any installed software. This module also produced a useful negative result: the Networkwalks Hash Calculator was unable to extract a hash from My Locked PDF1.pdf, which meant the Password Cracker could not be run against that file through this route. Rather than a failure of the overall approach, this outcome illustrated a real limitation of relying on a single online extraction tool, and reinforced why having a second, local method available — as used in Module 1 is valuable when a web-based tool cannot handle a particular file.

Across both modules, the exercise reinforced an important security lesson: file passwords that are short, common, or found in publicly available wordlists such as [rockyou.txt](#) can be cracked quickly, regardless of whether the underlying protection mechanism (in this case, PDF encryption) is itself strong. The strength of a password matters as much as the strength of the encryption protecting it, and using a password found in a common wordlist provides very little real protection against a determined attacker with access to the file.

From a defensive standpoint, this highlights why individuals and organizations should use long, unique, randomly generated passwords for sensitive documents rather than dictionary words, names, or predictable patterns, and why password-protected files should not be relied upon as a sole security control for highly sensitive data.

